

Guía de Laboratorio: Control y Selección de Servomotres

Curso: Dinámica de Maquinaria

1. Objetivos del Laboratorio

- Comprender y aplicar la modulación por ancho de pulso (PWM) para el control de posición en servomotores.
- Implementar modelos matemáticos de interpolación (Spline Cúbico) para el control de perfiles de velocidad y aceleración en mecanismos.
- Familiarizarse con entornos de desarrollo profesionales (PlatformIO) para la programación de microcontroladores ESP32.
- Establecer criterios de ingeniería para la selección de actuadores rotativos.

2. Marco Teórico

Criterios de Selección de Servomotores

Para diseñar un mecanismo, la selección del actuador no debe ser empírica. Se deben evaluar tres parámetros fundamentales:

1. **Potencia Mecánica:** Los fabricantes suelen dar el torque (τ) en $\text{kg} \cdot \text{cm}$ y la velocidad en $\text{s}/60^\circ$. Se debe convertir la velocidad a rad/s (ω) y el torque a $\text{N} \cdot \text{m}$. La potencia mecánica requerida por el eslabón es $P = \tau \cdot \omega$.
2. **Factor de Seguridad (FS) de Potencia:** En dinámica, las inercias y las aceleraciones generan picos de torque que superan los cálculos estáticos. Se recomienda un factor de seguridad entre **1.5 y 2.0** sobre el torque nominal del motor para evitar el atasco mecánico (*stall*) y el sobrecalentamiento de la bobina.
3. **Voltaje de Alimentación:** El torque máximo de un servo depende directamente de su voltaje. Un SG90 ofrece mayor torque a 6,0V que a 4,8V. Es vital asegurar que la fuente de alimentación pueda entregar la corriente máxima de atasco (*stall current*) sin que el voltaje caiga, lo cual reiniciaría el microcontrolador si comparten la misma fuente.

Modulación por Ancho de Pulso (PWM)

Los servomotores estándar no se controlan variando el voltaje linealmente, sino mediante una señal cuadrada enviada a una frecuencia específica.

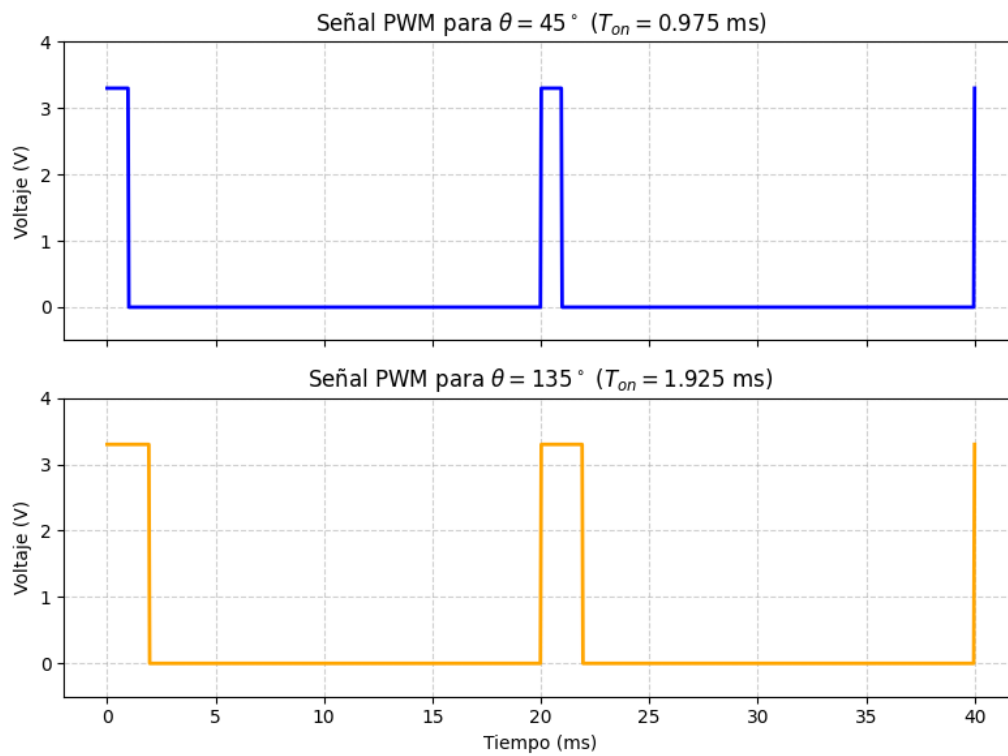


Figura 1: Relación entre el ancho de pulso PWM y la posición angular teórica del servomotor.

- **Frecuencia del Servo (50Hz):** Un servo SG90 espera recibir un pulso cada 20 milisegundos ($1/50\text{Hz} = 0,02\text{s}$).
- **Ancho de Pulso:** La posición angular depende de cuánto tiempo la señal se mantenga en estado ALTO (3,3V o 5V) dentro de esos 20ms.
- **Configuración en C++:** La función `motor.attach(pin, 500, 2400)` es crucial. Por defecto, las librerías asumen que el pulso va de $1000\mu\text{s}$ para 0° a $2000\mu\text{s}$ para 180° . Sin embargo, por tolerancias de fabricación, los SG90 reales requieren un rango más amplio. Los valores 500 y 2400 configuran el hardware de la ESP32 para garantizar el recorrido mecánico completo sin vibraciones en los topes.

Comunicación Serial

Para depurar el código y recibir datos del computador, se usa el protocolo UART. El comando `Serial.begin(115200)` establece la velocidad de transmisión en baudios (bits por segundo). Se escoge 115200 porque es un estándar rápido y estable para la arquitectura del chip ESP32, evitando cuellos de botella al imprimir datos dinámicos en tiempo real.

Planeación de Trayectorias: Spline Cúbico

Cuando se envía una orden de posición instantánea, la aceleración teórica es infinita, generando sacudidas destructivas en el mecanismo. Para evitarlo, controlamos la posición θ en función del tiempo t usando un polinomio de tercer grado.

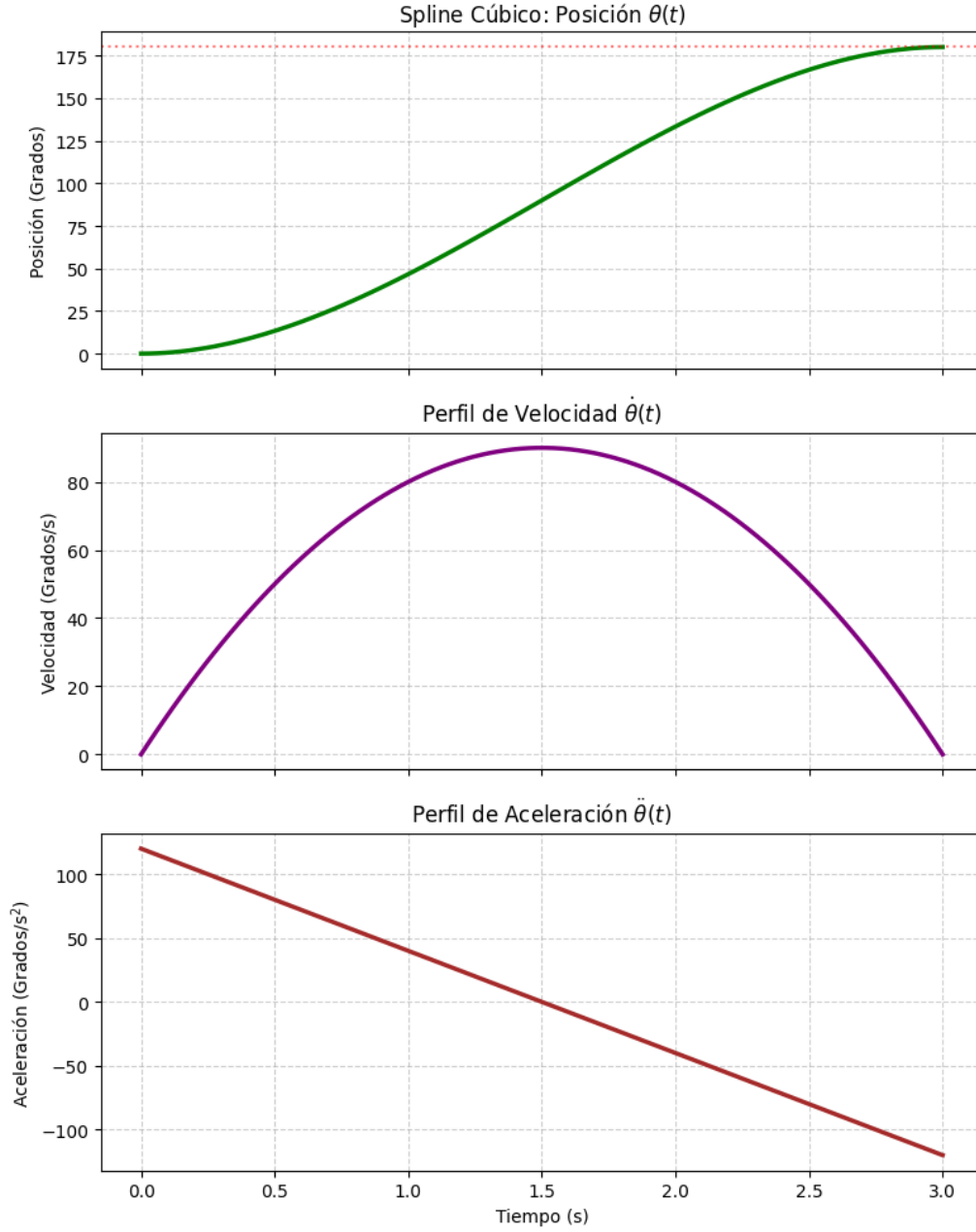


Figura 2: Perfiles cinemáticos de posición, velocidad y aceleración generados por un Spline Cúbico.

La ecuación general de trayectoria es:

$$\theta(t) = a_0 + a_1t + a_2t^2 + a_3t^3 \quad (1)$$

Para garantizar un movimiento suave, establecemos cuatro condiciones de frontera (reposo a reposo):

1. Posición inicial en $t = 0$: $\theta(0) = \theta_i \implies a_0 = \theta_i$
2. Velocidad inicial en $t = 0$: $\dot{\theta}(0) = 0 \implies a_1 = 0$
3. Posición final en el tiempo total t_f : $\theta(t_f) = \theta_f$
4. Velocidad final en t_f : $\dot{\theta}(t_f) = 0$

Resolviendo el sistema de ecuaciones, obtenemos los coeficientes dinámicos:

$$a_2 = \frac{3(\theta_f - \theta_i)}{t_f^2} \quad (2)$$

$$a_3 = \frac{-2(\theta_f - \theta_i)}{t_f^3} \quad (3)$$

El microcontrolador calculará posiciones intermedias en pequeños diferenciales de tiempo (dt) para "guiar" al motor.

3. Preparación del Entorno: PlatformIO

1. **Instalación:** Instalen Visual Studio Code (VS Code). En Extensiones, busquen e instalen **PlatformIO IDE**. Reinicien el editor.
2. **Crear Proyecto:** En PlatformIO (ícono de la hormiga), seleccionen *New Project*. Nombren su proyecto, en *Board* elijan **Espressif ESP32 Dev Module** y en *Framework* elijan **Arduino**.
3. **Configuración:** Abran el archivo `platformio.ini` y añadan:

```
monitor_speed = 115200
lib_deps = madhephaestus/ESP32Servo@^3.0.5
```

4. **Compilar y Cargar:** En la barra inferior de VS Code:
 - El **visto bueno** (✓) compila y verifica errores.
 - La **flecha** (→) compila y carga el código a la placa. *Si se queda en Connecting...*, mantengan presionado el botón "BOOT" en la ESP32.
5. **Formatear (Borrar Memoria):** Para limpiar la placa antes de cargar un nuevo `main.cpp`, vayan al menú lateral de PlatformIO → *esp32dev* → *Platform* → **Erase Flash**.

4. Códigos Base de Referencia

A continuación, se presentan dos códigos de ejemplo diseñados para controlar **un único servomotor**.

Nota para la actividad: Estos scripts son únicamente el punto de partida. Para aprobar las Actividades A y B, deberán analizar esta lógica y modificar la estructura para lograr controlar **dos servomotores de manera simultánea**.

Ejemplo Base 1: Control de Posición por PWM (Un Motor)

```
1 #include <Arduino.h>
2 #include <ESP32Servo.h>
3
4 Servo miServo;
5 const int pinServo = 4; // Asegurese de que el pin coincida con su
   cableado
6
7 void setup() {
8   Serial.begin(115200);
9   ESP32PWM::allocateTimer(0);
10  miServo.setPeriodHertz(50);
11  miServo.attach(pinServo, 500, 2400);
12  Serial.println("Configuracion terminada.");
13 }
14
15 void loop() {
16  miServo.write(0); // Posicion 0 grados
17  delay(1000);
18
19  miServo.write(90); // Posicion 90 grados
20  delay(1000);
21 }
```

Ejemplo Base 2: Control de Trayectoria con Spline (Un Motor)

```
1 #include <Arduino.h>
2 #include <ESP32Servo.h>
3
4 Servo miServo;
5 const int pinServo = 4;
6
7 void moverConSpline(Servo &motor, float theta_i, float theta_f, float
   tiempoTotal) {
8   float a0 = theta_i;
9   float a1 = 0.0;
10  float a2 = 3.0 * (theta_f - theta_i) / pow(tiempoTotal, 2);
11  float a3 = -2.0 * (theta_f - theta_i) / pow(tiempoTotal, 3);
12
13  int pasos = tiempoTotal * 50;
14  float dt = tiempoTotal / pasos;
15
16  for (int i = 0; i <= pasos; i++) {
17    float t = i * dt;
18    float theta_t = a0 + a1 * t + a2 * pow(t, 2) + a3 * pow(t, 3);
19
20    motor.write(theta_t);
21    delay(dt * 1000); // Pausa en milisegundos
22  }
23 }
24
25 void setup() {
26   Serial.begin(115200);
27   ESP32PWM::allocateTimer(0);
28   miServo.setPeriodHertz(50);
29   miServo.attach(pinServo, 500, 2400);
30 }
```

```

30 }
31
32 void loop() {
33     moverConSpline(miServo, 0, 90, 2.0); // 0 a 90 grados en 2 segundos
34     delay(500);
35     moverConSpline(miServo, 90, 0, 2.0); // 90 a 0 grados en 2 segundos
36     delay(500);
37 }

```

5. Actividades Prácticas

Paso 1: Montaje del Hardware

1. Conecten el pin VIN o 5V de la ESP32 a la línea roja (+) de la protoboard, y GND a la línea azul (-).
2. Conecten los cables de poder de ambos servos en la protoboard.
3. Conecten el cable de señal (Naranja/Amarillo) del Motor 1 al pin **D4** de la ESP32, y el del Motor 2 al pin **D5**.

Paso 2: Actividad A - Control Simultáneo (Respuesta al Escalón)

Objetivo: Desarrollar un código en `src/main.cpp` que mueva ambos motores al mismo tiempo, alternando entre 45° y 135° cada 2 segundos. Deberán instanciar dos objetos `Servo` y asignar dos temporizadores de hardware distintos (`allocateTimer`).

Paso 3: Actividad B - Control de Tiempo con Spline Cúbico

Objetivo: Formatear la ESP32 (Erase Flash) y programar un **nuevo** `main.cpp`. El objetivo es mover ambos motores simultáneamente desde 0° hasta 180° exactamente en **3 segundos** utilizando la teoría del Spline Cúbico.

Analicen la función `moverConSpline` del Ejemplo 2. Deberán adaptar la lógica del ciclo `for` para que calcule y envíe las posiciones a los dos motores dentro del mismo bucle de tiempo, garantizando la sincronía mecánica.